Principal components analysis of images via back propagation

Garrison W. Cottrell
University of California, San Diego
Computer Science and Engineering C-014
La Jolla, California 92093

Paul Munro
University of Pittsburgh,
Information Science Department
Pittsburgh, Pennsylvania 15260

ABSTRACT

The recent discovery of powerful learning algorithms for parallel distributed networks has made it possible to program computation in a new way, by example rather than algorithm. The back propagation algorithm is a gradient descent technique for training such networks. The problem posed to the researcher using such an algorithm is discovering how it did solve the problem if a solution is found. In this paper we apply back propagation to a well understood problem in image analysis, i.e., bandwidth compression, and analyze the internal representation developed by the network. The network used consists of nonlinear units that compute a sigmoidal function of their inputs. It is found that the learning algorithm produces a nearly linear transformation of a Principal Components Analysis of the image, and the units in the network tend to stay in the linear range of the sigmoid function. The particular transform found departs from the standard Principal Components solution in that near-equal variance of the coefficients results, depending on the encoding used. While the solution found is basically linear, such networks can also use the nonlinearity to solve encoding problems where the Principal Components solution is degenerate.

1. INTRODUCTION

In previous work[1], we showed that neural networks trained by the back propagation learning algorithm[2] can successfully perform a useful signal processing task, i.e., image compression. In this paper, we briefly review that work, describe some recent results using restricted connectivity and show empirically that a nonlinear network finds a projection onto the principal subspace generated by the first few principal components of the image (with noise in the first component only). In addition, in accordance with recent theoretical work by Baldi & Hornick[3], we give empirical evidence that a linear back propagation network does this exactly.

2. BACKGROUND

2.1 Back propagation

Parallel distributed processing (PDP) models consist of networks of simple neuronlike processing units connected by weighted links. The units communicate by passing real-valued activation in a fixed range across the links. Feed-forward PDP networks are arranged in layers, with activation only passing one way through the network. The outer layers correspond to input and output. A model of a particular function is created by picking a representation of the domain and range of the function as patterns of activation across input and output units. Units in the internal layers are referred to as hidden units, and in a successful network correspond to an *internal representation* of the input environment that is useful in producing the desired output. Supervised learning schemes are ones in which the network is given repeated examples of the desired input-output mapping, and the network adjusts the weights between units to produce the desired function. The network thus *programs itself* to solve the task.

Back propagation is a supervised learning scheme for feed-forward networks that operates by changing the weights between the units in such as a way as to reduce the mean-squared error in the output. An input pattern is presented to the network, and activation is propagated forward through the network to the output units. The correct output pattern is provided to the output units, allowing the computation of the error between the actual and desired output. Back propagation provides a rule for propagating the error signal back through the network from the output units, allowing the hidden units to change their weights to reduce the error. The surprising thing is that the rule for changing the weights is a purely local one, in the sense that every unit can determine its error through the connections that already exist in the network, without appealing to global information. The weight-changing rule is obtained by taking the partial derivative of the mean square error in the output with respect to the weights, giving a gradient descent algorithm for reducing the error.

In many cases the network will discover a solution to the problem that was unknown to the user. In doing so, it develops a useful internal representation of the input at the intermediate levels of the network. It is often difficult to analyze this representation because the networks are nonlinear, many units are involved, and the representations are highly distributed over the set of internal units. The present research makes a first step towards unraveling the nature of these representations by applying the learning mechanism to a domain where the types of useful representations have been well studied.

2.2 The encoder problem

In PDP terms, image compression is a type of *encoder problem*[4] In encoder problems, a network is trained to perform an identity mapping over some set of inputs. The network is constrained to perform this mapping through a narrow channel of hidden units, forcing it to develop an efficient encoding in that channel. There are two interesting aspects to this: (a) the network is developing a compact representation of its "environment"; and (b) although the training algorithms were developed as supervised learning schemes, this problem can be regarded as unsupervised since the teaching signal is the same as the input. The system self-organizes to encode the environment.

2.3 Image compression

The transform encoding approach to image compression involves several stages: (1) A linear transform of the input to a set of less correlated coefficients, (2) quantization of the coefficients, and finally, (3) some encoding scheme which assigns a code word to the quantized coefficients. In this work, we ignore the third stage and use a naive quantizer. The optimal transform in terms of minimizing the mean square error (MSE) in the image is the Principal Components, or Hotelling transform[5]. This transform finds the eigenvectors of the covariance matrix of the image vectors (called the principal components), and produces the coordinates of the data along the eigenvectors with the highest eigenvalues. That is, it transforms the coordinate system to axes aligned with the highest variance of the data. Only the coordinates along the axes of highest eigenvalues are sent. For the number of axes (or coefficients) used, this provably minimizes the MSE. In our model, the number of axes or coefficients corresponds to the number of hidden units.

In reporting results, the MSE is normalized with respect to the average squared intensity of the image (abbreviated NMSE). We express this figure in percent. We measure image compression in terms of the bits per pixel (bpp), the number of bits actually transmitted over the channel per pixel of the output image. Although one might think it should be the ratio of bits in the original image to transmission bits, this can be a misleading measure. If the original image uses 15 bits/pixel, the final image could use significantly less without any apparent degradation, due to limitations in the resolving power of the human eye. Hence we use bpp, which is independent of the number of bits used for the original pixels.

3. PROCEDURE

The network used for image compression is shown in Figure 1. It takes a square patch of the image, passes it through the hidden units, and reproduces it on the output. Various ratios of inputs to hiddens may be used to achieve different compression rates. Where necessary, we will specify this. Each unit at the hidden and output layers computes the sum of its inputs weighted by the connection strengths, and then "squashes" this through the nonlinear logistic function shown in Figure 2. The logistic used in this work has a range of [-1,1]. The input gray scale (ranging from 0 to 255) was scaled linearly into this range. It was found that the more the problem was restricted to the upper half of the linear range of the logistic, the better the network performed, indicating that this is basically a linear problem.

During training, the input to the network was drawn randomly from the image shown in Figure 3. The network was trained for 50,000 iterations as a learning rate of .25 and another 50,000 at .01. A search for optimal learning rates and minimal number of iterations was not done. In order to achieve true compression, the outputs of the hidden units must be quantized. The squashing function forces all outputs into the range [-1,1], so no scaling is necessary. We simply used a uniformly spaced quantizer that retained the endpoints (-1 and 1). Quantization then consists of rounding the outputs of the hidden units to the nearest value (or quantization bucket). A 5 bit quantizer uses 32 values spaced evenly between the endpoints. Better results could be obtained by taking into account the output values most often used and using nonuniform bucket spacing. It turns out that in this problem, the hidden units rarely reach the outer limits of the squashing function, so the endpoints of the quantizer were wasted.

Once this network is trained, we have a patch compressor, and we can reproduce the whole image by tiling the original image with it (no overlapping tiles). The result of this procedure is that the image can be reproduced with some loss of information using less than 1 bit/pixel, representing an eight-fold compression of the information in the image. A reproduction of the original image using 5 bits of hidden unit output using a 144 x 12 x 144 network is shown in Figure 4. This corresponds to .87 bpp, with an NMSE of 0.537%. The network also does a good job of compressing several images it was not trained on[1].

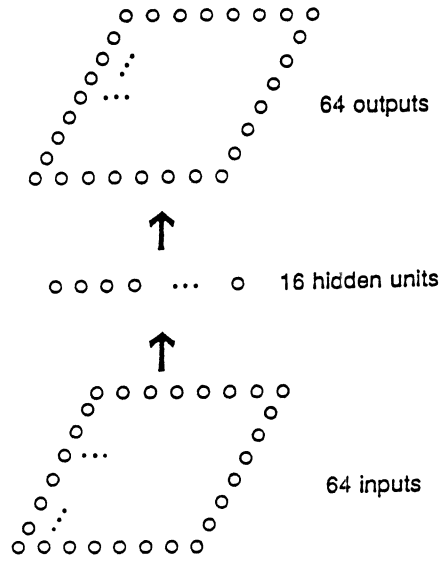

Figure 1. An example network. Different input/hidden ratios may be used.

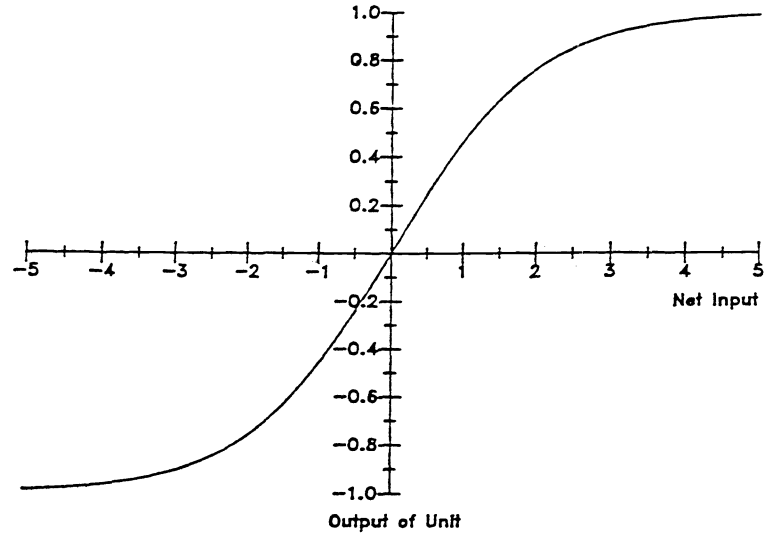

Figure 2. The logistic transfer function used by the hidden and output units.

In recent work with Doug Willen, we tried to see what would happen with a more plausible receptive field organization. In the human, each ganglion cell takes its input from a small subset of the entire retina. In this work, we considered the effects of limiting the receptive fields of the hidden units to 4 x 4 overlapping receptive fields, while maintaining the full output connectivity. This gives a measure of the information that can be extracted from a limited receptive field. In this case, the hidden units must predict their surround, and effectively combine their predictions. The result is that the NMSE in the output increases by only 30% using 11% of the input connections in the fully connected model. That is, the interacting predictions of the hidden units about their surround are sufficient to predict most of the information. This was borne out by further experiments restricting the output connections in the same way as the input connections, thus eliminating the predictive surround. This model resulted in over 100% more error, showing that the surround predictions were being put to use.

Figure 3. The digitized video image used for training the network.

Figure 4. The result of compressing the image: This represents .87 bpp.

4. ANALYSIS OF THE INTERNAL REPRESENTATION

Now the question is, how does it do it? What is the internal representation developed by the hidden units? The short answer is (roughly): The hidden units develop weight vectors that span the principal subspace of the image vectors. That is, they develop a "distributed" representation of the principal components. This result was obtained empirically by taking the eigenvectors corresponding to the image patch and giving them as input to the network. That is, (treating the network **N** as a "matrix"), we find **E'** such that:

$$\mathbf{E' = NE}$$

where **E** is the matrix formed by taking the eigenvectors of the image in descending order of eigenvalue as the columns. The resulting set of vectors is then multiplied by the transpose of the eigenvector matrix:

$$\mathbf{I' = E'E}$$

If the network spans the principal subspace of dimension **k**, then the resulting matrix **I'** will have **k** 1's on the diagonal and 0's everywhere else. This test was devised by Williams[6]. to test his symmetric error correction scheme for feature discovery. His network essentially found distributed principal components as well, but his algorithm was restricted to a linear network with symmetric weights, or a linear network with weight decay.

This test was thus devised for linear networks, but it can be used to see how the nonlinear network distorts the principal components. The upper left hand corner of the **I'** matrix for networks using two different encodings of the input is shown in Table 1. The network in Table 1(A) maps [0,255] into [0,1], and the one in (B) maps into [0,85]. Basically, the results show a noisy 1st component (brightness) and the rest of the components are slightly shortened. The noise in the first component is probably due to the fact that the output layer has to represent the highest values by entering the nonlinear range of the squashing function. This effect manifests itself in a reconstructed picture that is slightly dimmer than the original.

(A)

1.0	0.2	0.3	0.4	0.3
0.0	0.6	-0.1	-0.1	-0.1
-0.1	0.0	0.7	0.0	-0.1
0.0	0.0	0.0	0.7	0.0
0.0	0.0	0.0	0.0	0.6

(B)

1.0	0.2	0.2	0.3	0.3
0.0	0.8	0.1	0.1	0.1
0.0	0.0	0.8	-0.1	-0.1
0.0	0.1	0.0	0.8	0.0
0.0	0.0	0.0	0.1	0.7

(C)

1.0	-0.0	-0.0	-0.0	-0.0
-0.0	1.0	-0.0	-0.0	-0.0
-0.0	0.0	1.0	-0.0	-0.0
0.0	0.0	0.0	1.0	-0.0
0.0	0.0	-0.0	-0.0	1.0

Table 1. The upper left hand corner of **I'** for (A) the nonlinear network using a [0,1] encoding of the gray scale; (B) the non-linear network using [0,0.85] encoding; and (C) the linear network (using [0,1] encoding).

These results continue down to the thirteenth row, after which the entries are all nearly 0. The maximum possible is 16 rows from a network with 16 hidden units. It is interesting to note that we can use back propagation in a linear network as well. The results from that test are in Table 1(C). Here the principal components are faithfully reproduced without error. It also interesting that these components are found in order - by the end of 5000 training cycles, the network produces an **I'** that shows it spans the first 7 principal components. After 10000, it spans the first 9. This makes sense given that back propagation is trying to minimize the mean square error - this is the best way to go about it.

We set up some correspondences between our network and the usual image compression system. Recall that the first step in transform encoding is to multiply the patch vector by a matrix to obtain less correlated coefficients:

$$\mathbf{y = Ax}$$

The analog in our network is that **A** is the weight matrix between the input and hidden unit layers, with each row of **A** corresponding to the input weights on one hidden unit, and each hidden unit outputs a semilinear version of y_i.

Also, in reconstructing the image, in PCA the reconstruction is accomplished via the following transform:

$$x' = A^{-1}y$$

where A^{-1} is actually the transpose of **A** (recall, however that **A** is not a square matrix, since not all principal components are used). This means that a PCA network is *symmetric*: the hidden units are connected to the output patch by the same weights they are connected to the corresponding input patch. The reconstructed image is a linear combination of the columns of A^{-1}; these are called the *basis images*. In our network, the basis images are the projective field of the hidden units (the output weights of each unit), passed through the nonlinear squashing function.

An actual principal components analysis of the image would have each incoming weight vector correspond to one principal component. This would be a "localist" representation of the principal components. In this case, the outputs of the hidden units would have variance equal to the eigenvalue of their eigenvector[5]. These eigenvalues, and the variance of the hidden unit outputs during image reconstruction, are shown in Table 2 for the three networks of Table 1. From this table, it is easy to see that while the eigenvalues differ by several orders of magnitude, back propagation tends to distribute the variance much more evenly over the hidden units. As the encoding moves into the linear range, the differences in variance increase. Even in the case of the linear network, however, where the outputs of the hidden units are free to vary, the difference in variance from highest to lowest is only one order of magnitude. We hypothesize that this should minimize the effects of channel errors since no unit is crucial to the reproduction of the image. If an error is suspected, the unit's output could be replaced by its average value. An interesting research issue is to determine what is the cause of this pressure to spread the variance evenly. Baldi & Hornick[3] suggest that this is a matter of chance: The possible initial random weights leading to a localist PCA solution (and thus high variance separation) are a set of measure 0.

[0,1]	[0,0.85]	Linear	Eigenvalues
0.12734	0.09780	0.64923	2.93256
0.12459	0.09421	0.37990	0.11210
0.12112	0.09200	0.24748	0.10025
0.12042	0.09126	0.21718	0.03848
0.11999	0.09099	0.20477	0.03234
0.11763	0.08652	0.19823	0.02729
0.11661	0.08601	0.17211	0.01942
0.11542	0.08369	0.16134	0.01431
0.11533	0.08006	0.14223	0.00900
0.11305	0.07827	0.13278	0.00635
0.11144	0.07363	0.09653	0.00601
0.10821	0.07201	0.09061	0.00531
0.10504	0.06579	0.07473	0.00399
0.10390	0.05407	0.07310	0.00285
0.10347	0.04180	0.06673	0.00255
0.09896	0.03267	0.06585	0.00247

Table 2. A comparison of the variance of the hidden unit outputs during image reconstruction for the nonlinear networks, the linear network, and the eigenvalues for the image.

The variance of the coefficients is strongly correlated with the amount of image variance along the axis the hidden unit reflects. We ran a simulation where during the reconstruction process, we recorded the mean square error of the resulting image if each hidden unit's output was set to 0. The results of this experiment for a 16 hidden unit network showed the resulting MSE in terms of gray scale ranging from 295 to 459.

The fact that the network does not find a localist representation of the components is also apparent upon visual inspection of the weight matrix. Figure 5 shows the weight matrix for each of ten hidden units from the 144 x 25 x 144 network thresholded at various levels, one hidden unit per row. The last column corresponds to the basis images[5] learned by the network. An interesting point here is that they look more alike than different in their level of complexity. That is, there is not a clear ordering as there is for the basis images of the Hotelling transform.

The aim of this research was to investigate the nature of internal representations developed by the back propagation algorithm in the case of autoencoding. It was reassuring to find that when the problem is basically linear, back propagation finds a basically linear solution. The advantage of the nonlinearity comes out when the problem is nonlinear. In the encoder problem first investigated in the recent PDP literature[4], the network is given an environment where one unit at a time in the input vector is on. Rumelhart et al.[2] showed that given such an environment a back propagation network can learn the binary encoding of the position of the unit in order to turn on the corresponding unit in the output. In this case, the principal

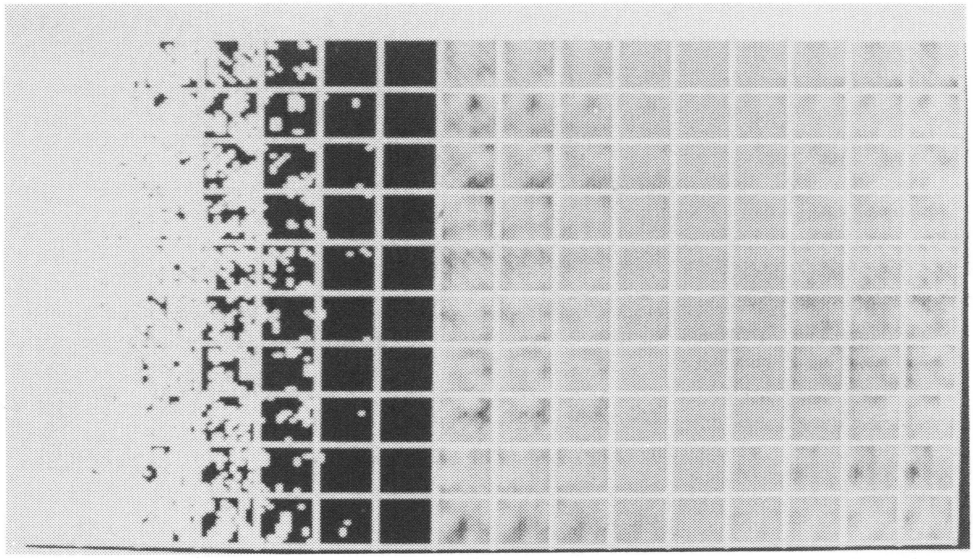

Figure 5. Each row corresponds to one hidden unit. The left hand 7 columns represent the input weights thresholded from -0.75 to +0.75. The output weights are similar, indicating the network is achieving symmetric connectivity. The right hand columns are the result of driving the output patch with that hidden unit alone, at levels running from -1 to 1. This gives an indication of the dynamic range of the nonlinear basis images.

components solution is degenerate; all of the eigenvalues are equal. Yet back propagation can use the nonlinearity in the network to solve the problem. Thus back propagation demonstrates optimality in a variety of situations only some of which may be addressed by purely linear models.

5. CONCLUSION

One way to describe the solutions discovered by back-propagation learning is to say that the network discovers regularities in the input. This report adds a new vocabulary for discussing the kinds of regularities discovered in the case of autocoding. We can say that back propagation is finding the principal components of the input. We can look at the variance of the hidden units as indicative of their usefulness in the resulting solution.

The major result of this work is in demonstrating the efficacy of current connectionist techniques for programming by example rather than algorithm. We have termed this *extensional programming*[1]. The results presented here suggest that this technique is a powerful one. Its naive application to a problem of current interest among engineers resulted in respectable performance compared to current techniques. This suggests that one useful approach to problems for which no algorithm is known, or for which no parallel algorithm is known, is to use connectionist representations of the problem and allow the network to discover the program itself. Analysis of the internal representations thus discovered may aid in our understanding of the problem and lead to methods for doing the programming ourselves. A variety of problems with which cognitive science is concerned with are of this character - the input-output behavior is known, but the algorithm is not. With extensional programming we can begin to investigate algorithms that we did not ourselves invent.

6. REFERENCES

1. Cottrell, G., Munro, P. and Zipser D. (1987) Learning internal representations from gray-scale images: An example of extensional programming. In Proceedings of the Ninth Annual Cognitive Science Society Conference, Seattle, Wa.

2. Rumelhart, D. E., Hinton, G.E., and Williams, R.J. (1986). Learning representations by back-propagating errors. Nature, 323, 533-536.

3. Baldi, Pierre, and Hornick, Kurt (1988). Neural networks and principal components analysis: Learning from examples without local minima. Working paper, Department of Mathematics, University of California at San Diego.

4. Ackley, D. H., Hinton, G. E., & Sejnowski, T.J. (1985). A learning algorithm for Boltzmann machines. Cognitive Science, 9, 147-169.

5. Gonzales, R.C., & Wintz, P. (1977). Digital image processing, Reading, MA: Addison-Wesley.

6. Williams, R.J. (1985). Feature discovery through error correction learning (Tech. Rep. 8501). La Jolla: University of California at San Diego, Institute for Cognitive Science.